

Technical Report – Phase 1

Members: Seth Garciano, Tristin Bates, Troy Franks and Yisong Wang

Date: October 6, 2026

1. Organization and Design Objectives

TotallyRealEstate is a hypothetical real estate company located in Alberta that has 8 internal employees as well as 100 independent agents. They currently use spreadsheets and applications to hold records pertaining to things such as clients, properties, listings, showings, potential offers and sale records. This system creates duplicates with inconsistent links.

Phase 1 aims to propose an EER model with a relational schema that has been normalized to 3NF to provide a consistent database structure that TotallyRealEstate can use. This model will contain 11 tables that will cover the records listed previously, those being clients, agents, staff, properties, listings, showings, bookings, offers, and completed sales.

Scope and Deliverables

Deliverable	Report location	Companion files
EER diagram	Section 4, three views	Editable source and three exports in EER_diagrams
3NF relational schema	Sections 5–6	Schema, foreign keys and normalization in Relational_Schema
Complete dictionary	Appendices A–F	All 11 tables and 63 attributes in the corrected dictionary
Technical report	This document	Decisions, assumptions, approval conditions, contributions and AI disclosure
GitHub organization	Section 9	Root README and required folder structure

2. Design Choices and Assumptions

ID	Description	Assumption	Reason
A01	Property and listing Identity	PROPERTY and LISTING are separate, meaning a property could be relisted	Physical characteristics, price, dates and status of a listing have different life cycles within the database
A02	Joint buyers and sellers	Listings may have joint sellers, offers may have joint buyers, and a client may have both roles	LISTING_SELLER and OFFER_BUYER represent participation without having mutually exclusive client categories
A03	Party capacity	Each showing allows up to three active bookings, with 1 to 3 visitors per booking.	No configurable capacity fields are required because these limits are fixed
A04	Companion registration	Each booking records one lead client while additional visitors are counted in party_size	Only the lead client's contact information needs to be stored; companions do not need their own records
A05	Active bookings and history	PENDING and CONFIRMED bookings are active. CANCELLED and REJECTED bookings remain as history	Capacity should only apply to any active bookings while keeping past booking records
A06	Showing before an offer	An OFFER can reference a LISTING directly without requiring a SHOWING	A showing and an offer are separate business events, so an offer wouldn't require a prior showing
A07	Responsible agents	Listings, showings, and offers each have their own responsible agent	The agent responsible for a listing may differ from an agent that is handling a showing or offer
A08	Staff classification	Internal staff and independent agents are stored separately, with STAFF using a role_code	The staff roles don't have unique attributes requiring a subtype hierarchy

3. Business Rules and Model Constraints

The following business rules define the main constraints of TotallyRealEstate's database.

Rule	Business Rule	Model Location
BR01	Clients, agents, staff, properties, listings, showings, bookings, offers and sales have unique identifiers	Primary keys and NOT NULL constraints

BR02	Each listing belongs to one property and one agent, while properties and agents can have multiple listings	PROPERTY/AGENT 1 to LISTING 0..N. Mandatory FKs
BR03	A listing may have multiple sellers. A draft listing can have none. Non-draft listings require at least one seller	LISTING_SELLER and participation constraint
BR04	A property can have at most one ACTIVE or UNDER_OFFER listing	Conditional uniqueness of LISTING.property_id
BR05	Asking price has to be positive, and listing dates must be valid	LISTING validation rules
BR06	Each showing belongs to only one listing and one agent and can have 0 or more bookings	SHOWING FKs and SHOWING_BOOKING relationships
BR07	A showing has to end after it starts, and a non-cancelled showing can't overlap for the same property/agent	SHOWING date and time constraints
BR08	Each booking belongs to one showing and one lead client and has 1 to 3 visitors	SHOWING_BOOKING FKs and party_size constraint
BR09	A showing can have at most 3 active bookings	Conditional booking count
BR10	A client can have at most one active booking for the same showing	Conditional uniqueness on showing_id and client_id
BR11	Each offer belongs to one listing and one submitting agent and has at least one buyer	OFFER FKs and OFFER_BUYER relationship
BR12	Offer amounts have to be positive and offer dates must be valid dates	OFFER validation rules
BR13	A listing can have at most one ACCEPTED offer	Conditional uniqueness on OFFER.listing_id
BR14	A sale must reference an accepted offer, and an offer can lead to at most one sale	SALE_TRANSACTION FK and unique offer_id
BR15	Sale price must be positive and completion can't occur before offer acceptance	SALE_TRANSACTION validation
BR16	The sale coordinator must be active staff with the COORDINATOR role.	STAFF eligibility constraint
BR17	Seller/listing and buyer/offer combinations can't be duplicated	Composite primary keys
BR18	Cancelled and closed records are retained, and referenced records cannot be deleted or have their primary keys changed.	Status fields and RESTRICT rules

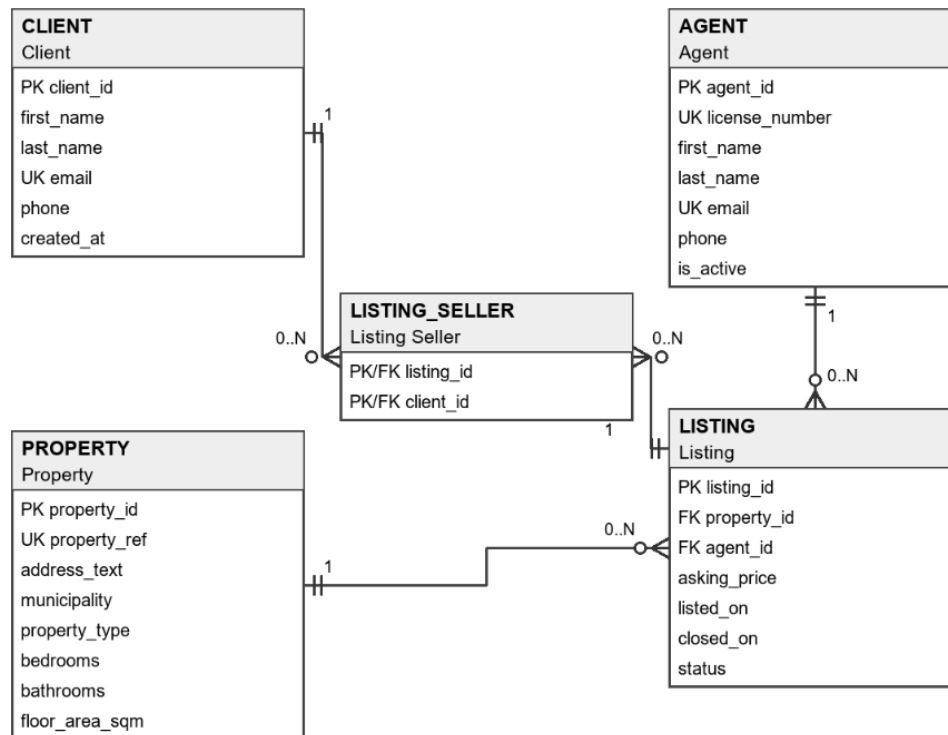
Constraint Representation

Primary keys, foreign keys, uniqueness constraints, and validation rules represent most business requirements. Conditional and cross-table rules, like active booking limits and accepted offer requirements, are documented as business rules because they depend on other records.

4. EER Diagram

The EER diagrams use crow's-foot notation and show primary keys, foreign keys, cardinality, participation and relevant constraints

EER Property and Listing



BR03 Non-draft listings require at least one seller

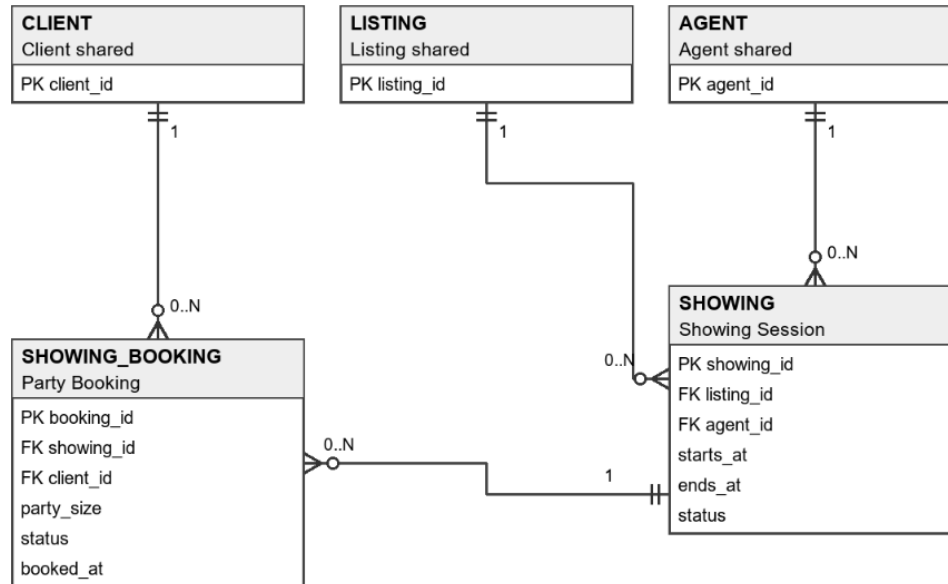
1 exactly one 0..1 optional one 0..N optional many 1..N one or more

PK primary key FK foreign key UK unique Shared attributes: View 1

Listings reference one property and one agent. Sellers are represented through LISTING_SELLER

EER Showing and Booking

Showings may have multiple bookings, with booking capacity and party size rules documented in section 3



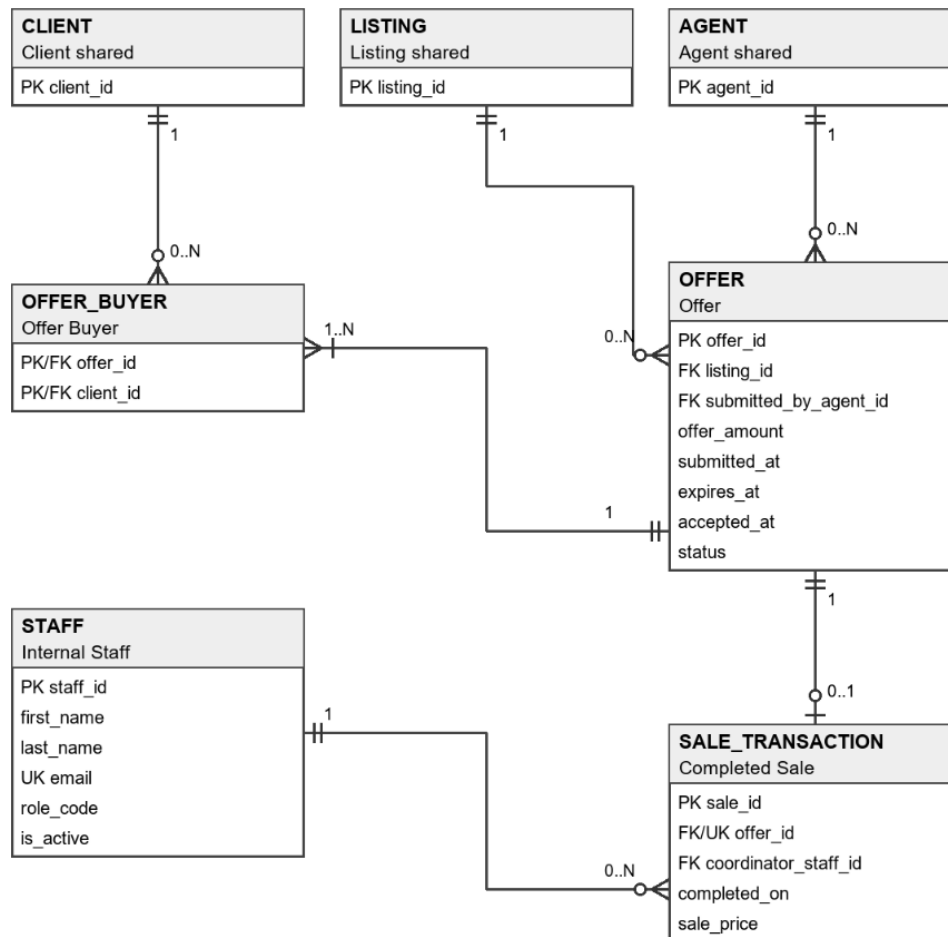
BR09 Up to 3 active bookings per session; 1–3 visitors per party

1 exactly one 0..1 optional one 0..N optional many 1..N one or more

PK primary key FK foreign key UK unique Shared attributes: View 1

EER Offers and Sales

Offers may have multiple buyers and may lead to one completed sale



BR13 At most one accepted offer per listing

1 exactly one 0..1 optional one 0..N optional many 1..N one or more

PK primary key FK foreign key UK unique Shared attributes: View 1

5. Relational Schema and Mapping

Entities map to tables with the same names. LISTING_SELLER and OFFER_BUYER convert many-to-many participations into just two one-to-many relationships and use composite primary keys. SHOWING_BOOKING is an associative entity with its own identifier, party size, status and booking timestamp

CLIENT (*client_id* PK, *first_name*, *last_name*, *email* UK, *phone*, *created_at*)

AGENT (*agent_id* PK, *license_number* UK, *first_name*, *last_name*, *email* UK, *phone*, *is_active*)

STAFF (*staff_id* PK, *first_name*, *last_name*, *email* UK, *role_code*, *is_active*)

PROPERTY (*property_id* PK, *property_ref* UK, *address_text*, *municipality*, *property_type*, *bedrooms*, *bathrooms*, *floor_area_sqm*)

LISTING (*listing_id* PK, *property_id* FK, *agent_id* FK, *asking_price*, *listed_on*, *closed_on*, *status*)

LISTING_SELLER (*listing_id* PK/FK, *client_id* PK/FK)

SHOWING (*showing_id* PK, *listing_id* FK, *agent_id* FK, *starts_at*, *ends_at*, *status*)

SHOWING_BOOKING (*booking_id* PK, *showing_id* FK, *client_id* FK, *party_size*, *status*, *booked_at*)

OFFER (*offer_id* PK, *listing_id* FK, *submitted_by_agent_id* FK, *offer_amount*, *submitted_at*, *expires_at*, *accepted_at*, *status*)

OFFER_BUYER (*offer_id* PK/FK, *client_id* PK/FK)

SALE_TRANSACTION (*sale_id* PK, *offer_id* FK/UK, *coordinator_staff_id* FK, *completed_on*, *sale_price*)

OFFER does not store showing_id. SALE_TRANSACTION does not repeat property_id, listing_id or client identifiers. An offer references its listing, and a sale references its offer; the property and participating buyers and sellers can be traced through those relationships.

Referential Integrity and Attribute Constraints

Foreign key	Referenced key	On delete	On update
LISTING.property_id	PROPERTY.property_id	RESTRICT	RESTRICT
LISTING.agent_id	AGENT.agent_id	RESTRICT	RESTRICT

LISTING_SELLER.listing_id	LISTING.listing_id	RESTRICT	RESTRICT
LISTING_SELLER.client_id	CLIENT.client_id	RESTRICT	RESTRICT
SHOWING.listing_id	LISTING.listing_id	RESTRICT	RESTRICT
SHOWING.agent_id	AGENT.agent_id	RESTRICT	RESTRICT
SHOWING_BOOKING.showing_id	SHOWING.showing_id	RESTRICT	RESTRICT
SHOWING_BOOKING.client_id	CLIENT.client_id	RESTRICT	RESTRICT
OFFER.listing_id	LISTING.listing_id	RESTRICT	RESTRICT
OFFER.submitted_by_agent_id	AGENT.agent_id	RESTRICT	RESTRICT
OFFER_BUYER.offer_id	OFFER.offer_id	RESTRICT	RESTRICT
OFFER_BUYER.client_id	CLIENT.client_id	RESTRICT	RESTRICT
SALE_TRANSACTION.offer_id	OFFER.offer_id	RESTRICT	RESTRICT
SALE_TRANSACTION.coordinator_staff_id	STAFF.staff_id	RESTRICT	RESTRICT

All foreign keys are NOT NULL. Referenced parent rows cannot be deleted or have their primary keys changed. Business records maintain stable IDs, with status values indicating cancellation or closure.

Constraint type	Requirement in this model
Composite PK	LISTING_SELLER(listing_id, client_id); OFFER_BUYER(offer_id, client_id).
Ordinary uniqueness	Property reference, agent licence number, registered emails and SALE_TRANSACTION.offer_id.
Conditional uniqueness	property_id for active listings; listing_id for ACCEPTED offers; (showing_id, client_id) for active bookings.
Ranges and dates	Positive amounts; 1–3 visitors; non-negative room counts; end after start; allowed states and consistent dates.

6. Functional Dependencies and 3NF Verification

Table	Main functional dependency	Key notes
CLIENT	client_id → first_name, last_name, email, phone, created_at	Non-null emails are conditionally unique, client_id identifies all rows
AGENT	agent_id → license_number, first_name, last_name, email, phone, is_active	license_number and email are additional candidate keys

STAFF	staff_id → first_name, last_name, email, role_code, is_active	email is an additional candidate key
PROPERTY	property_id → property_ref, address_text, municipality, property_type, bedrooms, bathrooms, floor_area_sqm	property_ref is an additional candidate key
LISTING	listing_id → property_id, agent_id, asking_price, listed_on, closed_on, status	PK determines all non-key attributes
LISTING_SELLER	listing_id + client_id → no non-key attributes	All-key association. No non-key dependency
SHOWING	showing_id → listing_id, agent_id, starts_at, ends_at, status	PK determines all non-key attributes
SHOWING_BOOKING	booking_id → showing_id, client_id, party_size, status, booked_at	Active client/session uniqueness is not a whole table candidate key
OFFER	offer_id → listing_id, submitted_by_agent_id, offer_amount, submitted_at, expires_at, accepted_at, status	PK determines all non-key attributes
OFFER_BUYER	offer_id + client_id → no non-key attributes	All-key association. No non-key dependency
SALE_TRANSACTION	sale_id → offer_id, coordinator_staff_id, completed_on, sale_price	offer_id is an additional candidate key

1NF: Every attribute stores a single value, no lists of client IDs.
2NF: Single-column primary keys have no partial dependencies, and the two composite-key association tables only hold key attributes.
3NF: Client contacts, agent details, and property characteristics live in their own tables. Everything else just references them or stores its own facts.

How Normalization Changes Design

Listings and Joint Sellers

The initial design stored seller information as repeating lists, violating 1NF. 1NF separates sellers into individual rows. 2NF separates listing and client attributes from the association. 3NF moves property and agent details into their own tables, producing LISTING, CLIENT, PROPERTY, AGENT and LISTING_SELLER.

Showings and Party Bookings

The original showing record had a repeating list of party bookings. For 1NF to be followed, we turned each booking into its own row with `booking_id`, `showing`, `lead client`, `party size`, `status` and `booking time`. For 2NF, nothing needs splitting as `booking_id` is a single-column key with no partial dependencies. For 3NF, we take out showing details, like session times and responsible agents, and the client details, like name and phone, into their own tables, leaving just `SHOWING`, `SHOWING_BOOKING` and `CLIENT`. Agent info stays in the separate `AGENT` table. Party size, status and `booked_at` stay in `SHOWING_BOOKING` since they depend on only the booking itself, and a new booking after a cancellation would just get a new `booking_id`.

Offers, Joint Buyers and Sales

For offers, we would start by expanding the list of buyers into separate rows to satisfy 1NF. For 2NF, the offer amount and dates would move into the `OFFER` table, and the client details would go into `CLIENT`, so `OFFER_BUYER` only keeps the composite key. 3NF leaves the listing, property and submitting agent details inside their own tables. `OFFER` would just store the references to the listing and agent. A sale is treated as its own fact with `sale_id` as the primary key and `offer_id` as a unique foreign key, and the final sale amount doesn't have to match the original offer price. `AGENT`, `STAFF` and `PROPERTY` already have their own single-column keys and don't contain any more dependencies, so no further splits are needed.

7. Model Fit and Design Boundaries

Separating properties from listings keeps the history of when that same property gets listed again. Separating sessions from bookings allows us to track the lead client, party size, booking capacity and cancellations. Separating offers from sales means we can record any accepted offers that may not be finished yet. The association tables handle joint sellers and buyers, and a client's role comes from which table they appear in.

`STAFF` stores the current role as `BROKER`, `ADMIN`, `COORDINATOR`, `ACCOUNTANT` or `MARKETING`. `AGENT` is only used for independent agents and their license details. The scope doesn't need any role-specific attributes or subtypes. Therefore, there is no

supertype/subtype hierarchy in this version. The same person may be a buyer and a seller at the same time.

Supported Questions and Limitations

The model can support queries about a property's listing history, agent sessions, client bookings and who is buying or selling, as well as the offers and sales for any listing. Nothing forces a showing to happen before an offer is made. This version keeps the current states and the business records we need to keep only, not every single change over time. It leaves out unnecessary things like staff role history, agent handovers, offer revisions, rentals, commission payments and big open houses. Those can be added later once we know exactly which data is required.

Keys stop the same identifier from being used twice, but they can't catch when the same property somehow ends up with two different references. The email uniqueness assumption and identity verification need team review. Any rules about conditional counts, who participated and cross-table state constraints are recorded in Section 3 instead of being forced into ordinary attribute CHECK constraints.

8. Approval Conditions and Later Data Scope

Later requirement	Choice in the proposal	Effect on the current model
Large source	Calgary Historical Property Assessments (Parcel), 2021–2025.	Keep public assessments separate from synthetic TRE business data.
Spark processing	Apache Spark/PySpark on Databricks Free Edition.	Retain source, batch, year and clean grain.
Warehouse	Delta dimensions and facts; Databricks SQL Warehouse.	Later analytical structures use year, community, and property type.
BI	Power BI Desktop.	Report assessed value trends, community comparisons, and property-type distribution.
Cloud database	Neon PostgreSQL.	Use the same core relational schema, not two independently updated authoritative stores.

NoSQL extension	Cloud Firestore listing descriptions and showing notes.	Link through stable listing_id or showing_id; core amounts and states remain in PostgreSQL.
-----------------	---	---

9. Project Repository and Contributions

Project Repository: https://github.com/SethG1262/Advanced_DataBase

The repository contains the Phase 1 deliverables. Member contributions are recorded below.

Phase 1 tools: The EER model is maintained in editable draw.io (diagrams.net) format. Microsoft Word is used to prepare the technical report and data dictionary, while GitHub stores the design files and records team contributions. PostgreSQL 18 is the target database for the relational schema.

Phase 1 files are organized as follows:

README.md

Phase1_Modelling/EER_diagrams/ Editable draw.io source and three PNG exports

Phase1_Modelling/Relational_Schema/ Schema and normalization notes

Phase1_Modelling/Data dictionary/ Complete table and attribute definitions

Phase1_Modelling/Technical Report/ Technical report

Contribution Table

Member	Phase 1 role	Actual specific tasks	Evidence
Troy Franks	Data dictionary	Creating and filling data dictionary	GitHub push
Seth Garciano	Relational Schema and normalization	Converting the EER diagram into the relational schema and normalizing it from 1NF-3NF	GitHub push
Tristin Bates	Technical Report	Creating, formatting, and structure of the technical report.	GitHub push
Yisong Wang	Business rules and EERD	Improve business rules and design database tables and create EER diagrams.	GitHub push

AI Use Disclosure

ChatGPT/Codex assisted with basic brainstorming, syntax support and translation between languages.

Appendix A CLIENT and AGENT

Client: One row represents one client (a buyer or seller) registered with TotallyRealEstate.

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
client_id	Unique identifier	Integer	PK	NOT NULL	1001
first_name	First name of client	Varchar		NOT NULL	John
last_name	Last name of client	Varchar		NOT NULL	Doe
email	Contact email	Varchar	UK	Optional; UNIQUE	Johndoe@gmail.com
phone	Contact phone	varchar		NOT NULL	403-123-4567
created_at	When client account was created	timestampz		NOT NULL	2026-10-06 10:00-06:00

Agent: One row represents an independent real estate agent affiliated with TotallyRealEstate.

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
agent_id	Unique identifier	Integer	PK	NOT NULL	1001
license_number	Agents' real estate licence	Varchar	UK	NOT NULL; UNIQUE	RE-ag-123
first_name	first name of agent	Varchar		NOT NULL	Agent
last_name	Last name of agent	Varchar		NOT NULL	bob
email	Contact email	varchar	UK	NOT NULL; UNIQUE	Agentbob@gmail.com
phone	Contact phone	varchar		NOT NULL	403-123-4567
is_active	Is the agent active	boolean		NOT NULL	true

Appendix B STAFF and PROPERTY

Staff: One row represents an internal, full-time administrative or operations employee at TotallyRealEstate.

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
staff_id	Unique identifier	Integer	PK	NOT NULL	1001
first_name	Staff's first name	Varchar		NOT NULL	Staff
last_name	Staff's last name	Varchar		NOT NULL	member
email	Contact email	Varchar	UK	NOT NULL; UNIQUE	Staffmember@gmail.com
role_code	Code that specifies role	Varchar		NOT NULL; see roles below	COORDINATOR
is_active	Current employment status	boolean		NOT NULL	true

role_code: BROKER, ADMIN, COORDINATOR, ACCOUNTANT or MARKETING.

Property: One row represents a unique real estate asset managed in the system

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
property_id	Unique identifier	integer	PK	NOT NULL	1001
address_text	Property street address	Varchar		NOT NULL	67 tickle boulevard
property_ref	Municipal property code	Varchar	UK	NOT NULL; UNIQUE	A-123
municipality	City where property is located	Varchar		NOT NULL	Calgary
property_type	Type of property	Varchar		NOT NULL; see types below	CONDO
bedrooms	Amount of bedrooms	integer		Optional; integer >= 0	2
bathrooms	Amount of bathrooms	numeric		Optional; >= 0	2
floor_area_sqm	Total square metres in property	numeric		Optional; > 0	92.90

property_type: DETACHED, CONDO, TOWNHOUSE or OTHER.

Appendix C LISTING and LISTING_SELLER

Listing: One row represents an individual market listing for a property

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
listing_id	Unique identifier	integer	PK	NOT NULL	1001
property_id	Foreign key referencing the property	integer	FK	NOT NULL	1234
agent_id	Foreign key referencing the agent	integer	FK	NOT NULL	6769
asking_price	Initial or current listed sale price	numeric		NOT NULL; > 0	650,000
listed_on	Date the listing went on the market	date		NOT NULL	2026-10-06
closed_on	Date the listing was sold or withdrawn	date		Optional; >= listed_on	2026-11-06
status	Status of listing	varchar		NOT NULL; see states below	SOLD

status: DRAFT, ACTIVE, UNDER_OFFER, SOLD, WITHDRAWN or EXPIRED. DRAFT, ACTIVE and UNDER_OFFER have NULL closed_on. SOLD, WITHDRAWN and EXPIRED require closed_on. Each property has at most one ACTIVE or UNDER_OFFER listing. Non-draft listings require at least one seller. Foreign keys: property_id -> PROPERTY.property_id; agent_id -> AGENT.agent_id.

Listing seller: One row represents an associative link attaching one seller client as an owner of a property listing

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
listing_id	Foreign key identifying listing	integer	PK/FK	NOT NULL	1001
client_id	Foreign key identifying client	integer	PK/FK	NOT NULL	1234

Composite primary key: (listing_id, client_id). Foreign keys: listing_id -> LISTING.listing_id; client_id -> CLIENT.client_id.

Appendix D SHOWING and SHOWING_BOOKING

Showing: One row represents a scheduled viewing appointment time slot hosted by an agent for a property listing

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
showing_id	Unique identifier	Integer	PK	NOT NULL	1001
listing_id	Foreign key identifying listing	Integer	FK	NOT NULL	1234
agent_id	Foreign key identifying agent	Integer	FK	NOT NULL	6769
starts_at	Date and time the showing begins	timestampz		NOT NULL	2026-10-06 14:00-06:00
ends_at	Date and time the showing ends	timestampz		NOT NULL; > starts_at	2026-11-06 14:30-07:00
status	Outcome of showing	varchar		NOT NULL; see states below	SCHEDULED

status: SCHEDULED, COMPLETED or CANCELLED. Non-cancelled showing intervals (starts_at, ends_at) must not overlap for the same property or agent. Foreign keys: listing_id -> LISTING.listing_id; agent_id -> AGENT.agent_id.

Showing Booking: One row represents a client's specific booking for a property showing appointment

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
booking_id	Unique identifier	integer	PK	NOT NULL	1001
showing_id	Foreign key identifying showing	Integer	FK	NOT NULL	1234
client_id	Foreign key identifying lead client	Integer	FK	NOT NULL	6769
party_size	Amount of people viewing property	integer		NOT NULL; 1 to 3; includes lead client	3
status	Outcome of booking	varchar		NOT NULL; see states below	CONFIRMED
booked_at	Time the showing is booked at	timestampz		NOT NULL	2026-10-10 10:00-06:00

status: PENDING, CONFIRMED, CANCELLED or REJECTED. PENDING and CONFIRMED bookings are active. Each showing allows at most 3 active bookings; each showing/client pair has at most 1 active booking. Foreign keys: showing_id -> SHOWING.showing_id; client_id -> CLIENT.client_id.

Appendix E OFFER and OFFER_BUYER

Offer: One row represents a formal purchase proposal submitted by an agent for a property listing

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
offer_id	Unique identifier	integer	PK	NOT NULL	1001
listing_id	Foreign key identifying listing	integer	FK	NOT NULL	1234
submitted_by_agent_id	Foreign key identifying agent	integer	FK	NOT NULL	6768
offer_amount	Proposed purchase amount	numeric		NOT NULL; > 0	650,000
submitted_at	Timestamp when offer was sent	timestamptz		NOT NULL	2026-10-06 10:00-06:00
expires_at	Timestamp when offer expires	timestamptz		Optional; > submitted_at	2026-11-06 18:00-07:00
status	Current processing status of the offer	Varchar		NOT NULL; see states below	ACCEPTED
accepted_at	Timestamp when the offer was accepted	timestamptz		Only for ACCEPTED; see dates below	2026-10-21 10:00-06:00

status: SUBMITTED, ACCEPTED, REJECTED, WITHDRAWN or EXPIRED. Each offer requires at least one buyer. Each listing allows at most one ACCEPTED offer. All other offer states have NULL accepted_at. accepted_at must be >= submitted_at and <= expires_at when an expiry is present. Foreign keys: listing_id -> LISTING.listing_id; submitted_by_agent_id -> AGENT.agent_id.

Offer buyer: One row represents an associative link attaching one buyer client to a submitted purchase offer

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
offer_id	Foreign key identifying offer	integer	PK/FK	NOT NULL	1001
client_id	foreign key identifying client	integer	PK/FK	NOT NULL	1234

Composite primary key: (offer_id, client_id). Foreign keys: offer_id -> OFFER.offer_id; client_id -> CLIENT.client_id.

Appendix F SALE_TRANSACTION

Sale transaction: One row represents a completed real estate sale finalized from an accepted purchase offer

Attribute	Description	Data type/Format	Key	Validation rules	Sample/Default
sale_id	Unique identifier	Integer	PK	NOT NULL	1001
offer_id	Foreign key identifying offer	integer	FK/UK	NOT NULL; UNIQUE	1234
coordinator_staff_id	Foreign key identifying staff	integer	FK	NOT NULL	6769
completed_on	official date of transaction	date		NOT NULL; not before offer acceptance date	2026-11-06
sale_price	Final price of the sale	numeric		NOT NULL; > 0	650,000

The offer must be ACCEPTED. The coordinator must be active STAFF with role_code COORDINATOR. Completion uses the Calgary-local acceptance date. All foreign keys use ON DELETE RESTRICT and ON UPDATE RESTRICT. Cancelled and closed records are retained. Foreign keys: offer_id -> OFFER.offer_id; coordinator_staff_id -> STAFF.staff_id.